

AUTENTICAZIONE

Leonardo Querzoni
querzoni@dis.uniroma1.it



SAPIENZA
UNIVERSITÀ DI ROMA

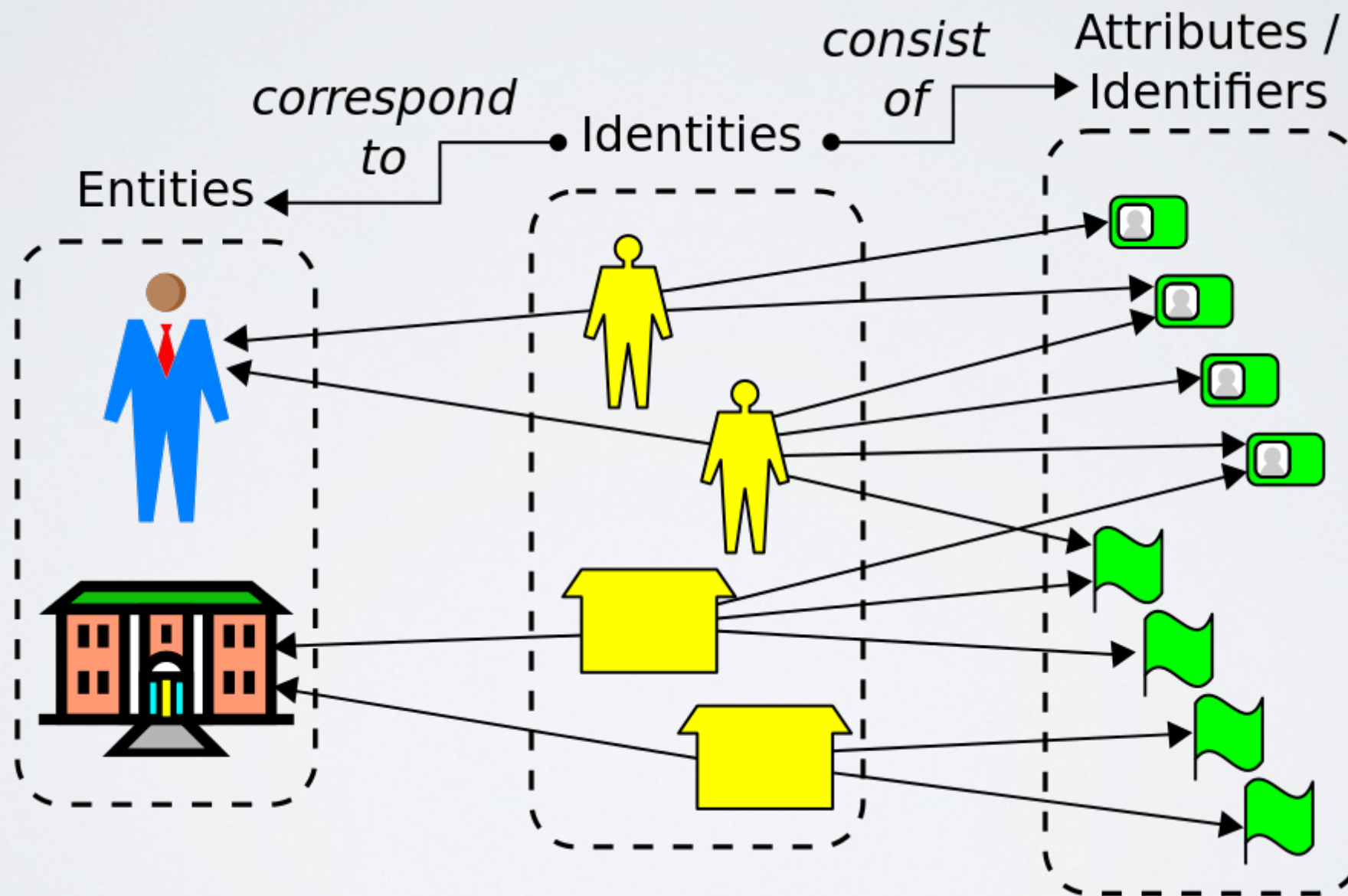


CIS SAPIENZA
CYBER INTELLIGENCE AND INFORMATION SECURITY

DIPARTIMENTO DI INGEGNERIA
INFORMATICA AUTOMATICA E
GESTIONALE ANTONIO RUBERTI

IDENTITÀ DIGITALI

Le identità digitali sono il modo in cui ogni individuo è rappresentato nel mondo virtualizzato dei sistemi software.



AUTENTICAZIONE

L'autenticazione è l'operazione con cui una identità digitale viene associata ad un individuo in modo che questo tramite essa possa eseguire delle operazioni.

- **Identificazione**: dichiarare in modo univoco l'identità digitale tramite cui si intende interagire con il sistema.
- **Verifica**: fornire una prova della corrispondenza tra l'identità digitale richiesta e la propria persona.

Autenticazione = Identificazione + Verifica

AUTENTICAZIONE

La fase di verifica tipicamente può avvenire con tre diverse modalità:

- Something you **know**
- Something you **have**
- Something you **are**

L'**autenticazione a due fattori** consiste in una verifica con due modalità contemporanee

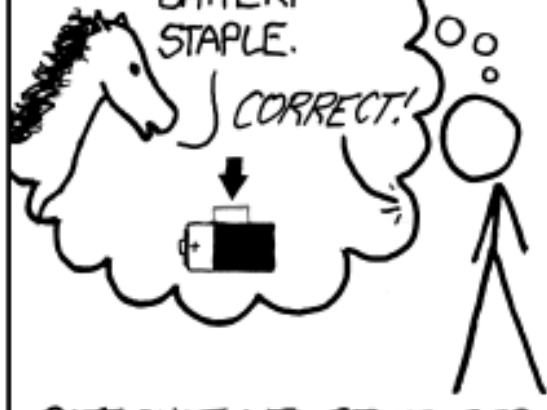
- Es: password + SIM telefono
- Es: bancomat + PIN

PASSWORD MANAGEMENT

La tipologia di verifica più comune è tramite password

- Le password più brevi di 10 caratteri sono deboli
- Le password più brevi di 20 caratteri solo minuscoli sono deboli
- Una buona password dovrebbe contenere:
 - Almeno un numero
 - Almeno un simbolo
 - Almeno una lettera maiuscola
 - Non contenere termini del dizionario

PASSWORD MANAGEMENT

□□□□□□□□□□□□□□□□ UNCOMMON (NON-GIBBERISH) BASE WORD ORDER UNKNOWN Tr0ub4dor &3 CAPS? □ COMMON SUBSTITUTIONS □□□ NUMERAL □□□ PUNCTUATION □□□□ (YOU CAN ADD A FEW MORE BITS TO ACCOUNT FOR THE FACT THAT THIS IS ONLY ONE OF A FEW COMMON FORMATS.)	~28 BITS OF ENTROPY □□□□□□□□ □□□□□□□□ □□□ □□□□ $2^{28} = 3 \text{ DAYS AT } 1000 \text{ GUESSES/SEC}$ (PLAUSIBLE ATTACK ON A WEAK REMOTE WEB SERVICE. YES, CRACKING A STOLEN HASH IS FASTER, BUT IT'S NOT WHAT THE AVERAGE USER SHOULD WORRY ABOUT.) DIFFICULTY TO GUESS: EASY	WAS IT TROMBONE? NO, TROUBADOR. AND ONE OF THE 0s WAS A ZERO? AND THERE WAS SOME SYMBOL...  DIFFICULTY TO REMEMBER: HARD
correct horse battery staple □□□□□□ □□□□□□ □□□□□□ □□□□□□ □□□□□□ □□□□□□ □□□□□□ □□□□□□ FOUR RANDOM COMMON WORDS	~44 BITS OF ENTROPY □□□□□□□□□□ □□□□□□□□□□ □□□□□□□□□□ □□□□□□□□□□ $2^{44} = 550 \text{ YEARS AT } 1000 \text{ GUESSES/SEC}$ DIFFICULTY TO GUESS: HARD	THAT'S A BATTERY STAPLE. CORRECT!  DIFFICULTY TO REMEMBER: YOU'VE ALREADY MEMORIZED IT

THROUGH 20 YEARS OF EFFORT, WE'VE SUCCESSFULLY TRAINED
EVERYONE TO USE PASSWORDS THAT ARE HARD FOR HUMANS
TO REMEMBER, BUT EASY FOR COMPUTERS TO GUESS.

PASSWORD MANAGEMENT

Risultato

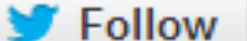
- Gli utenti usano sempre la stessa password per tutti i servizi
 - Le password uniche sono difficili da ricordare
- Le password sono generalmente deboli
- Le password non vengono mai cambiate (neanche dopo essere state compromesse!)
- Gli utenti si appuntano le password su carta

PASSWORD MANAGEMENT

Risultato



 **Xavier**
@xavez

  Follow

“password reset is the new login”

 Reply  Retweeted  Favorited  More

2 RETWEETS **3** FAVORITES

8:14 AM - 25 Jul 13

Reply to @xavez

 **Rhythmus.be** @rhythmvb
@xavez oAuth.io
Details

30 Jul

PASSWORD MANAGEMENT

Strategie consigliate

- Scegliere password complesse
 - sandwiching + padding
- Layering: password con complessità legata al loro uso
- Mai usare la stessa password due volte
- Usare un password manager

PASSWORD STORAGE

Problema: come tenere traccia delle identità degli utenti

- Database
- Directory

Come memorizzare la password in modo da garantirne la confidenzialità?

PASSWORD STORAGE

Soluzione 1

- Memorizzare la password “in chiaro” nel database

Svantaggi:

- In caso di compromissione del database tutte le password sono visibili

PASSWORD STORAGE

Soluzione 2

- Memorizzare *sha1* (password) nel database

Pro:

- Le password non sono immediatamente visibili

Svantaggi:

- Vecchie funzioni di hash come md5 e sha1 non sono più considerate sicure
- Un cluster con 25 GPU nel 2012 generava 348 miliardi di hash al secondo.
Oggi ?
- Rainbow tables attack

PASSWORD STORAGE

Soluzione 3

- Memorizzare *sha1*(fixed_secret + password) nel database

Pro:

- Le rainbow tables sono inutili

Svantaggi:

- Se il fixed_secret è lo stesso per tutte le password prima o poi qualcuno lo scoprirà
- Ogni attacco lavora contemporaneamente su tutte le password

PASSWORD STORAGE

Soluzione 4

- Memorizzare *sha1*(per_password_salt + password) nel database

Pro:

- Ogni attacco può lavorare su una singola password alla volta

Svantaggi:

- Non fornisce una soluzione al potere computazionale continuamente crescente a prezzi sempre più bassi

PASSWORD STORAGE

Soluzione 5

- Memorizzare *bcrypt*(per_password_salt + password) nel database

Pro:

- Soluzione ad oggi ottimale

[protected form] = [salt] + protect([protection func], [salt] + [credential])

Altre funzioni: Argon2, PBKDF2, scrypt

PASSWORD STORAGE

BCrypt

- random salt + fattore di costo + password
- Utilizza il salt per trasformare la password in una chiave
 - La complessità di questa fase è regolata dal fattore di costo
- Utilizza la chiave ottenuta per codificare una stringa nota
- La stringa codificata rappresenta la versione sicura della password

Vantaggi:

- Se le CPU diventano più veloci basta aumentare il fattore di costo
- BCrypt è inerentemente lento
- BCrypt non è ad oggi ottimizzabile per il funzionamento su GPU/ASIC/FPGA

AUTENTICAZIONE CON OAUTH

OAuth

